Constraint Satisfaction Problem Assignment

CSA17 Artificial Intelligence | Unit II Searching Techniques
Answers to Questions 4 and 12

Q4 University Exam Timetabling

Task i CSP formulation

A constraint satisfaction problem consists of variables, a domain of possible values for each variable, and constraints that restrict permitted combinations. A solution is a complete exam timetable that satisfies every hard constraint.

For each course i, define $X_i = (T_i, R_i, P_i)$, where T_i is its start time, R_i is its assigned hall and P_i is its invigilator or faculty team. The exam duration d_i and enrolled student set S_i are input data. If faculty teams are fixed, P_i is a parameter rather than a decision variable.

The domain $D(X_i)$ contains all permitted time, hall and staff combinations. Remove combinations that violate hall capacity, required accessibility features, examination dates, or staff availability before search begins. This model assumes one suitable hall per exam; large exams can instead use a set of halls with sufficient total capacity.

Hard constraint	Meaning in the timetable
Student conflicts	If S_i and S_j share a student, their occupied time intervals must not overlap.
Hall resources	A hall cannot host overlapping exams, and its capacity must meet enrolment.
Required order	If course i must precede course j, finish(i) must be no later than start(j).
Faculty availability	Every assigned staff member must be available throughout the exam and cannot supervise overlapping exams.
Special accommodations	Provide required accessible halls, equipment, assistance and approved extra time. Check conflicts using the affected student's actual exam interval.

For two intervals, non-overlap means $\text{finish}(i) \leq \text{start}(j)$ or $\text{finish}(j) \leq \text{start}(i)$. With equal, non-overlapping exam slots this reduces to $T_i \neq T_j$ for conflicting courses. Longer exams and extra-time arrangements require the interval model.

Preferences such as spreading a student's exams across days are soft constraints. They can be represented by penalty scores to minimise after hard feasibility is secured. A preference must never override a compulsory accommodation or resource limit.

Q4 Search and scalable scheduling

Task ii Backtracking search

Backtracking constructs a timetable one exam at a time. It selects an unassigned exam, tries an available placement, checks the constraints against earlier assignments, and continues if the placement is consistent. If an exam has no legal placement, it undoes the previous assignment and tries another value. A complete consistent assignment is returned; exhaustion of every branch proves infeasibility for the model.

Consider three equal-duration slots S1, S2 and S3 in chronological order, halls H1 and H2, and exams A, B and C. Assume A and B share students, A must precede C, and C can use only S2 because of faculty availability. All halls have sufficient capacity and H2 is accessible for B.

If the search first places A in S2, C has no permitted later slot. Backtracking removes that choice and tries A = (S1, H1). It can then place C = (S2, H1) and B = (S3, H2). This satisfies the student conflict, precedence, hall and accommodation requirements. The example assumes all other staff are available.

Task iii Constraint propagation and forward checking

Forward checking removes inconsistent values from the domains of unassigned neighbouring exams after each assignment. After A is placed in S1, it removes every S1 placement from B because they share students. It also removes C placements that violate A-before-C and removes H1 bookings that overlap A. If any remaining domain becomes empty, the branch fails immediately.

In the failed choice A = S2 above, C initially has domain {S2}. The precedence rule removes S2, leaving an empty domain. Forward checking therefore rejects A = S2 before assigning B or exploring its alternatives. Every removed value must be restored when the search backtracks.

MRV selects the exam with the fewest remaining placements: C, with only S2, is the most restricted in this example. The degree heuristic breaks an MRV tie by selecting the exam constraining the most unassigned exams. LCV tries a placement that removes the fewest values from neighbouring domains, preserving future scheduling flexibility.

Forward checking examines the effect of the current assignment. Maintaining arc consistency can propagate restrictions further between unassigned variables, detecting some failures that forward checking misses, at a greater processing cost.

Task iv Real world challenges and scalability

Large enrolment lists create many conflicts; limited halls, staff absences and extra-time needs reduce the available choices. Incorrect student data or contradictory precedence requirements can make the timetable impossible. A late change can also disrupt many already published exam arrangements.

A practical strategy is to validate the input, build a student-conflict graph, filter domains, and use backtracking with MRV, degree tie-breaking and propagation in a constraint programming solver. Use LCV where its calculation saves enough search effort. Global resource constraints can represent hall and staff capacities efficiently.

Independent groups may be solved separately only when they share no students, rooms, staff or other linking constraints. For very large instances, first find a feasible timetable and then improve soft penalties with local search or large neighbourhood search. Repair only the affected part after a late change, keep fixed commitments where possible, and recheck all hard constraints. This combines reliable feasibility with practical speed; worst-case search remains exponential.

Q12 Mobile Tower Frequency Assignment

Task i Graph colouring CSP model

Represent the towers by an undirected graph G = (V, E). Each tower is a variable X_v with domain {1, 2, 3}, representing three frequency bands. Add an edge (u, v) exactly when the two towers have overlapping coverage. Every edge imposes X_u ≠ X_v. A valid assignment colours every tower without giving adjacent towers the same band.

Use six towers A, B, C, D, E and F. The edges are AB, AC, BC, BD, CD and DE. Tower F is an isolated tower in a remote valley. The diagram is a constraint graph, so the line lengths do not represent geographic distances.

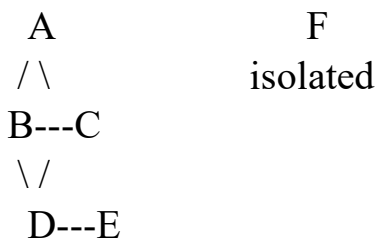


Figure 1 Six tower constraint graph with one independent tower

Tower	Neighbours	Degree	One valid band
A	B, C	2	1
B	A, C, D	3	2
C	A, B, D	3	3
D	B, C, E	3	1
E	D	1	2
F	None	0	1

Every listed edge has different bands at its endpoints: AB = 1–2, AC = 1–3, BC = 2–3, BD = 2–1, CD = 3–1 and DE = 1–2. F can use any band because it has no neighbours. Triangle ABC requires three different bands, so two bands cannot solve this graph. The displayed three-band assignment proves that its chromatic number is exactly three.

Task ii MRV and degree heuristic

Minimum remaining values (MRV) chooses the unassigned tower with the smallest current domain. Initially all six domains have three values. The degree heuristic breaks this tie by choosing a tower with the largest number of unassigned neighbours. B, C and D each have degree three; choose B as a consistent tie-break.

Assign B = 2. Forward checking leaves A = {1, 3}, C = {1, 3}, D = {1, 3}, E = {1, 2, 3} and F = {1, 2, 3}. MRV selects among A, C and D. Their numbers of unassigned neighbours are 1, 2 and 2 respectively; degree therefore favours C or D. Choose C = 3, then A and D are forced to 1, E may use 2 or 3, and F remains unrestricted.

Q12 Value ordering and early failure

Task ii Applied least constraining value example

Least-constraining value (LCV) orders a selected tower's legal bands by how few values they remove from unassigned neighbours. It orders values, while MRV and degree order variables. At the initial state all bands are symmetric, so LCV cannot distinguish them without further restrictions.

For a separate illustrative partial state on the same graph, suppose A = 1 and E = 2 are already fixed. Both assignments are consistent because A and E are not adjacent. Forward checking gives B = {2, 3}, C = {2, 3}, D = {1, 3} and F = {1, 2, 3}. MRV ties B, C and D, and each has two unassigned neighbours. Select B by alphabetical tie-break.

Candidate for B	Values removed from neighbours	LCV score
B = 2	Remove 2 from C; D does not contain 2.	1
B = 3	Remove 3 from C and 3 from D.	2

LCV therefore tries B = 2 first. C becomes {3}, while D remains {1, 3}. MRV then chooses C = 3 and forward checking forces D = 1. With F = 1, the complete assignment is A1, B2, C3, D1, E2, F1. LCV preserves more immediate options; it does not guarantee success in every CSP.

Task iii Forward checking detects a failure early

Use a different branch of the same graph with A = 1 and E = 1. This partial assignment is legal because the towers are not adjacent. The remaining domains of B, C and D are all {2, 3}. The following trace shows how a locally legal decision leads to an empty domain.

Stage	B domain	C domain	D domain
After A1 and E1	{2, 3}	{2, 3}	{2, 3}
Assign B2	Assigned 2	{3}	{3}
Try C3	Assigned 2	Assigned 3	Empty

When C receives band 3, edge CD removes 3 from D. D has already lost band 1 because E = 1 and band 2 because B = 2. Its domain becomes empty, so forward checking reports failure before attempting to assign D or the unrelated tower F. Checking only assigned neighbours would accept C = 3 temporarily, because C differs from A and B, and would discover the problem later.

Changing C is impossible because its only value was 3. After restoring domains, trying B = 3 forces C = 2 and again leaves D with no band. Thus the branch with A = E = 1 has no completion. Backtrack to E and try E = 2; then A1, B2, C3, D1, E2 is feasible. Domain restoration is essential so that rejected branches do not incorrectly remove values from later branches.

Q12 Structure and an additional band

Task iv Independent subproblems

A connected component has no edges to other components. In this example, {A, B, C, D, E} forms one component and {F} forms another. Solve the first component and assign F any band independently. The solutions can then be combined because the problem has only edge-based interference constraints. No global limit on use of a band is assumed.

More generally, if a graph separates into components with n1, n2 and further tower counts, solving each independently avoids exploring combinations between them. Instead of a naive search over 3^n assignments, independent exhaustive searches examine roughly a sum of 3^ni component assignments, although actual solver performance depends on graph structure and propagation.

Task iv Tree structured constraint graphs

A tree is connected and has no cycles. Root the tree, order nodes from the root to the leaves, and work backwards to remove parent values without a supporting child value. After this consistency pass, assign the root and then each child a value compatible with its parent. For a binary CSP, this takes $O(n\, d^2)$ time, where n is the number of variables and d is the maximum domain size, assuming constant-time constraint tests.

With unrestricted band domains and only inequality edges, a tree needs at most two bands: alternate bands by depth. For example, the tree path A–B–D–E can use 1, 2, 1, 2. This path is an illustration of a tree, not the full six-tower graph, which contains triangles ABC and BCD.

The full example cannot directly use the tree algorithm because it has cycles. However, condition on B: removing B from the remaining graph leaves the path A–C–D–E and isolated F. For each candidate band of B, remove that band from A, C and D, then solve the remaining tree and isolated node. This is cycle-cutset conditioning. It works especially well when only a small number of towers must be fixed to break all cycles.

Task v Availability of a fourth frequency band

Change each unrestricted domain from {1, 2, 3} to {1, 2, 3, 4}. The constraint graph and the rule $X_u \neq X_v$ stay the same. Every previous solution remains valid, and the additional choice may make previously impossible instances feasible. It is optional to use band 4 unless a separate requirement says otherwise.

In the failed branch A = E = 1, B = 2 and C = 3, D now retains {4} instead of becoming empty. Assigning D = 4 and F = 1 completes that branch. In the original graph, however, a fourth band is unnecessary for feasibility because a three-band solution already exists.

A fourth band does not guarantee a solution for every possible interference graph: five mutually overlapping towers form a complete graph on five vertices and require five bands. Restricted band availability or other operational constraints can also prevent feasibility.

The naive search space increases from 3^n to 4^n; for six towers, it grows from 729 to 4096 possible assignments. Nevertheless, extra flexibility can reduce backtracking in practice. MRV, degree, LCV and propagation remain useful, with their domain counts recalculated. If economical spectrum use is an objective, minimise the number of used bands after satisfying interference constraints; allowing four bands is different from requiring all four.